

Installing MessageFoundry — the engine + your configuration repository

Early Access · MessageFoundry v0.3.2 · September 2026

Install **MessageFoundry (MEFOR)** and create the **private Git repository** that holds your interface configuration. This guide accompanies [ADR 0017](#), which defines the deployment model.

Scope. This guide covers installation, the configuration repository, private Git, and multiple instances. For lab, shadow, limited, and full rollout stages, read the **Early-Adopter Installation & Rollout Guide**. That guide also covers security, PHI controls, reliability, and disaster recovery. For network exposure and TLS, read **DEPLOYMENT.md**. For the Windows service, read **SERVICE.md**.

1. The model in one paragraph

Install MessageFoundry as a **read-only, version-pinned Python wheel**. Keep Connections, Routers, and Handlers in a **separate private Git repository owned by your organization**. One configuration repository serves Test and Production, with optional POC or Staging instances. Each instance selects its environment and security settings at runtime. Developers review interface changes through pull requests. The engine remains a fixed dependency.

Keep these three areas separate:

Tier	What it is	Who owns / edits it
Engine	The installed <code>messagefoundry</code> wheel (pinned version)	The MessageFoundry project — you install it, never edit it
Config repo	Your private git repo: Connection/Router/Handler <code>.py</code> , code sets, <code>environments/<env>.toml</code> , fixtures	Your integration developers — authored via pull requests
Per-instance settings	<code>messagefoundry.toml</code> + <code>MEFOR_*</code> environment variables + secrets	Your operators, per deployed instance — never committed to the repo

2. Prerequisites

- **Python 3.14+** on each engine host (the engine requires 3.14+).
- **git**, plus a **private git host** — GitHub (private repo), GitLab, Azure DevOps, Bitbucket, or a self-hosted server. Nothing about MessageFoundry requires a public repo. Your config repo is yours.

- A source for the engine wheel: **public PyPI** is the distribution channel and the recommended install (`pip install "messagefoundry==<version>"`). If public-index installation is unavailable, use the signed GitHub Release wheel or a mirrored internal package index. Examples include Artifactory, Azure Artifacts, and private PyPI.
- Administrator/elevation on the host if you will install the engine as a Windows service (see [SERVICE.md](#)).

3. Step 1 — Install the engine (a pinned, read-only dependency)

On each host, create a virtual environment and install the engine at a **pinned version**:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1          # Linux/macOS: . .venv/bin/activate
pip install "messagefoundry==0.3.2"  # pin the exact version (core runtime only)
```

⚠ Early Access. MessageFoundry is beta-level software on public PyPI. The independent external review and penetration test recommended by ASVS Level 3 have not occurred. Pin the version that your organization has tested. Check [PyPI](#) for the current release. You can also install from your organization's private index.

Install only the extras that the host requires. `messagefoundry[postgres]` supports PostgreSQL.

`messagefoundry[sqlserver]` supports SQL Server and DATABASE connectors, with OS-level ODBC Driver 18.

`messagefoundry[harness]` supplies PySide6 only. The harness itself uses the separate `messagefoundry-harness` distribution. `messagefoundry[sftp]` supplies SFTP connectors. `messagefoundry[fhir]` supplies the FHIR codec and outbound. `messagefoundry[dicom]` supplies C-STORE SCP and its codec.

The install model supports these controls:

- **Non-editable.** A normal `pip install` lands the engine in `site-packages` as a regular installed package — not an editable checkout. Developers import its public surface. They do not have the engine source tree in front of them to change.
- **Pinned + reproducible.** The exact version is recorded in your config repo's `requirements.txt` (Step 2). An engine upgrade is a deliberate, reviewable one-line bump — never an accident. For a fully reproducible deploy, extend that into a hash-locked requirements file and install with `--require-hashes`.

Verify the release before you install (supply-chain integrity)

Verify the wheel's origin before installation. A pinned version or hash identifies a fixed file but does not establish who built it. Each release includes **SLSA build provenance** linking the wheel's SHA-256, source commit, and GitHub Actions builder, plus a **Sigstore signature**. Check both with the **GitHub CLI** (`gh ≥ 2.49`) and, optionally, `sigstore` (`pip install sigstore`). Install **only** the verified file.

```

$V = "0.3.2" # the version you intend to install – see PyPI for the current release

# Download the wheel + its Sigstore bundle from that release's assets
gh release download "v$V" --repo MEFORORG/MessageFoundry `
  --pattern "messagefoundry-$V-*.whl" --pattern "messagefoundry-$V-*.whl.sigstore*"

# Verify SLSA build provenance: artifact -> source commit -> builder workflow
gh attestation verify "messagefoundry-$V-py3-none-any.whl" --repo MEFORORG/MessageFoundry

# (defense in depth) Verify the Sigstore signature pins the release workflow identity
python -m sigstore verify identity "messagefoundry-$V-py3-none-any.whl" `
  --cert-identity "https://github.com/MEFORORG/MessageFoundry/.github/workflows/release.yml@refs/tags/v$V"
  --cert-oidc-issuer "https://token.actions.githubusercontent.com"

# Only if BOTH pass, install the exact file you verified
pip install ".\messagefoundry-$V-py3-none-any.whl"

```

The public PyPI wheel is byte-identical to the GitHub-built artifact, so the same attestation covers it. Download, verify, then install it:

```

$V = "0.3.2"
pip download "messagefoundry==$V" --no-deps -d .\verify
gh attestation verify (Get-ChildItem ".\verify\messagefoundry-$V-*.whl").FullName --repo
MEFORORG/MessageFoundry
pip install --no-index --find-links .\verify "messagefoundry==$V"

```

A substituted or relabelled file fails verification. Pair this check with `pip install --require-hashes -r requirements.lock` for a fully pinned deployment. Hash checks confirm that bytes match the lockfile. The identity check confirms who built them. Pre-releases tagged `-rc` also publish to production PyPI. The `--cert-identity` reference must match the installed tag, such as `refs/tags/v0.3.2`.

4. Step 2 — Create your config repo with the scaffolder

The `init` command creates a configuration repository that passes `check`:

```

messagefoundry init ./my-config-repo
cd my-config-repo

```

It writes:

```

my-config-repo/
├─ config/IB_EXAMPLE_ADT.py      # a runnable starter feed (receive ADT over MLLP → archive to file)
├─ environments/dev.toml        # non-secret per-environment values for env("...") lookups
├─ environments/prod.toml
├─ messages/sets/example_adt.h17 # a synthetic fixture (NO real PHI) that gates `check`
├─ messagefoundry.toml          # THIS instance's settings (environment + posture + store + API + egress)

```

```

├─ requirements.txt          # pins the engine: messagefoundry==0.3.2
├─ .github/workflows/check.yml # CI: install the pinned engine + run `messagefoundry check` on every
PR
├─ .vscode/settings.json     # points the VS Code extension at config/ + messages/
├─ .gitignore .gitattributes # excludes stores, secrets, captures, venvs, caches
└─ README.md

```

Validate it immediately:

```

pip install -r requirements.txt
messagefoundry check --config config --messages messages/sets

```

`check` validates the configuration and runs the synthetic fixture without delivery. Continuous integration runs the same check on each pull request. Replace the starter feed with your Connections, Routers, and Handlers.

The public engine API includes `inbound`, `outbound`, `@router`, `@handler`, `Send`, and `Message`. It also includes `MLLP`, `File`, `env`, and `code_set`. Configuration changes do not require engine edits.

5. Step 3 — Make it a *private* git repo

The scaffolded directory is ready to become your private repo:

```

git init
git add .
git commit -m "Initial MessageFoundry config (scaffolded)"
# then point it at your private host, e.g.:
git remote add origin git@github.com:your-org/mefor-config.git # a PRIVATE repo
git push -u origin main

```

Repository storage. Use the shared host from `git remote add` for multiple engine hosts, team access, or off-machine backup. A single-machine deployment without high availability can use a local repository. You can add a remote later. Options include self-hosted Forgejo, Gitea, or GitLab. You can also use your usual private host or a bare repository on a network share. For setup without a terminal, use **Set Up Version Control & Checks** or **Config Repo Storage Location**. Refer to `VERSION-CONTROL.md`.

Protect the repository with these controls:

- **Repo visibility.** Create the remote as **Private** on whatever host you standardize on. The engine imposes no requirement here — your config repo is yours, access-controlled by your git host's permissions (teams, SSO, branch protection).
- **No secrets, ever.** The scaffolded `.gitignore` already excludes `*.db`, `.env`, captures, and build craft. **Secrets — DB passwords, keys, credentials — are injected at runtime via `MEFOR_*` environment variables**, never written to the repo. The versioned `environments/<env>.toml` files hold only **non-secret** endpoints (hostnames, ports), which is exactly what you want diffable and reviewable.

- **No PHI in the repo.** Test fixtures are **synthetic only**. Real message bodies live in the engine's secured store at runtime, not in git.
- **CI as the gate.** The included `check.yml` installs the pinned engine and runs `messagefoundry check` on every PR. Enable branch protection on `main`. Require reviewed pull requests with passing checks.
- **Branch → PR → review → merge** is the entire authoring workflow. There is no separate "engine change" path for developers, because there is no engine to change.

6. Step 4 — Configure each instance (settings + posture + secrets)

Each deployed instance gets its own `messagefoundry.toml` (the scaffolder generates a starting one) plus its `MEFOR_*` environment. Two things every instance must state:

- `[ai].environment` — a free-form name (`test`, `prod`, `poc`, ...) that selects `environments/<name>.toml`.
- **Security posture, explicit and decoupled from the name:** `[security].handles_real_patient_data` (`true` | `false` — does this instance carry *real* PHI?) and `[security].production_instance` (`true` | `false` — is this a production tier?). Built-in names `dev` / `staging` / `prod` derive a sensible default posture. **any custom name must state posture explicitly** — the engine fails closed rather than guess.

Secrets and host-specific overrides come from the environment, e.g. `MEFOR_VALUE_<KEY>` for values used by `env("...")` in the graph, and `MEFOR_<SECTION>_<KEY>` for service settings. Precedence is **CLI flag > MEFOR_* env > messagefoundry.toml > built-in default**. Full reference: [CONFIGURATION.md](#).

7. Step 5 — Run / deploy an instance

```
messagefoundry serve --config config --env test --project-root C:\srv\mefor\my-config-repo
```

- `--project-root` (or `[environments].base_dir` in `messagefoundry.toml`) anchors `environments/<env>.toml` resolution to the repo root, so values resolve **regardless of the working directory**. This matters when the engine runs as a **Windows service under NSSM**, where the launch directory is not your repo. (Omit it only when you always launch from the repo root.)
- The engine **binds 127.0.0.1 by default** and **requires authentication**. To expose a channel off-loopback, configure **native TLS** (API: `[api].tls_cert_file` / `tls_key_file` or a trusted upstream terminator. MLLP inbound: per-connection `tls = true`) — a non-loopback bind without TLS is **refused at startup**. See [DEPLOYMENT.md](#).
- For production, run the engine as a **Windows service via NSSM** — see [SERVICE.md](#).
- For active-passive failover, refer to [CLUSTERING.md](#). Supply a floating VIP or L4 load balancer. Use a TCP-connect check for each inbound listener port so senders reach the primary after failover. MEFOR ships the clustering + the `/cluster/*` health-check endpoints, but **not** the load balancer itself. Single-node deployments need none of this.

Launching the admin console (in a browser)

Operators use the **browser web console** at `/ui`. Install its separate `messagefoundry-webconsole` wheel in the engine environment. The console has its **own version line**, rather than matching engine release numbers. At startup, the engine checks the pair's UI compatibility version and **refuses a mismatch**. Install a console release compatible with your pinned engine. The console is **on by default** for local instances:

```
pip install "messagefoundry-webconsole==0.2.15" # the /ui web console, into the same venv
# then (re)start the engine – there is no switch to turn on. To turn the console OFF, set
# [security].serve_web_console = false (the old [api].serve_ui spelling is refused at config load)
```

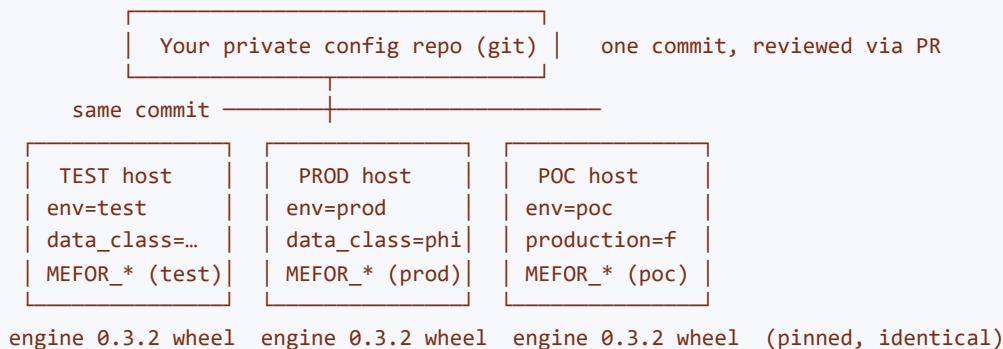
For local access, open `/ui` at `http://127.0.0.1:8765/ui` and sign in. The engine usually runs as a **service**.

For remote access, explicitly set `[security].serve_web_console = true` and configure TLS. Otherwise, an exposed instance serves only the JSON API and logs a warning. A declared TLS proxy also requires `[security].web_console_public_address`. Refer to `REMOTE-CONSOLE.md`.

The browser console replaces the retired PySide6 desktop console (BACKLOG #103). PySide6 now supports only the separate test harness. Install it with `pip install messagefoundry-harness`, then run `python -m harness`. The harness releases match the engine. The engine wheel excludes `harness/`. The `messagefoundry[harness]` extra supplies only PySide6, which the harness distribution installs automatically.

8. Multiple instances from one repo (Test, Production, POC...)

Deploy **the same reviewed configuration commit** to each instance. You do not need per-environment branches. Each host has its own `messagefoundry.toml` environment and security settings, plus its `MEFOR_*` variables:



Promotion = merge to `main` → deploy that commit everywhere. A Test instance resolves `environments/test.toml`. Prod resolves `environments/prod.toml`. A Test instance can never accidentally read Prod values. (Each instance is otherwise an independent engine with its own store. For multi-node high availability of a single instance, see `CLUSTERING.md`.)

9. Why integration developers can't modify the engine

Three layers reinforce the boundary:

1. **Packaging.** The engine is an installed, **non-editable** wheel. Developers work exclusively in the config repo. The engine source is not in their working tree.
2. **Version pinning.** `requirements.txt` pins an exact engine version. Each version change requires a reviewable pull request. Continuous integration reruns `check` against the new version before merge.
3. **(Optional) OS enforcement.** Operators can make the venv's `site-packages` read-only (Windows ACL) so the installed engine cannot be edited in place even by mistake.

Developers work in the configuration repository. Engine upgrades follow the review process below.

10. Upgrading the engine

1. Bump the pin in `requirements.txt` (e.g. `messagefoundry==X.Y.Z`) on a branch.
2. `pip install -r requirements.txt` and run `messagefoundry check` locally. Open a PR — CI re-validates your whole config against the new engine.
3. Merge, and roll the new commit to Test first, then Production. Because everything is pinned and your config is gated by `check`, upgrades are deliberate and reversible (pin back). The full upgrade/rollback runbook is in [EARLY-ADOPTER-GUIDE.md §13](#).

Where to go next

Topic	Reference
Staged install-to-production rollout, hardening, DR	EARLY-ADOPTER-GUIDE.md
The consumer deployment model (rationale)	ADR 0017
Service settings / environments	CONFIGURATION.md
Connections / the graph / <code>connections.toml</code>	CONNECTIONS.md
Windows service install	SERVICE.md
Network exposure / TLS	DEPLOYMENT.md
Security / auth / RBAC / audit	SECURITY.md
PHI handling / encryption	PHI.md
Multi-node high availability (needs a floating VIP / L4 LB)	CLUSTERING.md

Install a pinned engine, create a private configuration repository with `messagefoundry init`, and enable branch protection and continuous integration checks. Deploy reviewed commits to each instance with separate environment settings and secrets.

© 2026 **MessageFoundry, LLC** · MessageFoundry is licensed under **AGPL-3.0-or-later** · Document generated September 2026 for MessageFoundry v0.3.2.

MessageFoundry and the MessageFoundry word mark are trademarks of MessageFoundry, LLC. This document describes an **Early Access** release; check pypi.org/project/messagefoundry for the current version. Verify specifics against your own deployment and the engine's reference documentation.